FIAP · GLOBAL SOLUTION 2026.2 · CITIZENTECH

Rota Seca

API RESTful em Java com Spring Boot e Oracle para alertas de chuva por bairro, ocorrências de alagamento e pontos de apoio em Guarulhos.

INTEGRANTES (ORDEM ALFABÉTICA)

Arthur Albertini	RM 565954
Fabio Pereira Rogério Júnior	RM 564005
Gabriel dos Santos Cardoso	RM 562103
Lucas Oliveira Santos	RM 563617
Matheus Tonsa Martini	RM 564454

Engenharia de Software · 2º ano · Turma **2ESPF**
Equipe **NovaCore** · Solução **Rota Seca**

LINKS DA ENTREGA

GitHub: github.com/fajuniior/rota-seca-api
Deploy: rota-seca-api.onrender.com

DISCIPLINA

Domain Driven Design — Java

ÍNDICE

1 Descrição do projeto 2
2 Arquitetura da API 3
3 Diagrama de classes 4
4 Protótipos de tela 12
5 Tabela de endpoints 13
6 Modelo de dados (DER) 17
7 Regras de negócio, exceções e testes 20
8 Instruções de execução e deploy 22

Guarulhos, novembro de 2026 · Protótipo acadêmico: os dados do banco são simulados (nomes, telefones, ruas e locais fictícios). A Rota Seca não substitui as orientações da Defesa Civil (199).

1

5 pontos

Descrição do projeto

O problema

Em Guarulhos, bairros como Pimentas e Cocaia alagam com frequência nas chuvas de verão. Em entrevista feita pela equipe, uma moradora de 67 anos (persona **Neusa**) contou que já ficou presa num ônibus em uma avenida alagada, que o alerta oficial fala da cidade inteira e não da rua dela, e que não conseguiu falar com a irmã, que mora sozinha em outro bairro, quando a luz acabou.

As informações que ajudariam essas pessoas existem, mas estão espalhadas: a chuva medida pelos pluviômetros, o que os vizinhos estão vendo na rua, os locais abertos para esperar a chuva passar e a rede de familiares que poderia ligar para quem não viu o aviso.

A solução

A **Rota Seca** é uma CitizenTech que junta essas informações. Esta entrega é a **API RESTful** que serve o aplicativo: ela guarda e consulta os dados no Oracle da FIAP e aplica as regras que tornam o aviso confiável, como a confirmação por vizinhos, a validação pela Defesa Civil e a expiração de informações antigas.

Justificativa

- **Aviso no lugar certo:** o alerta é por bairro e a pessoa é avisada quando surge uma ocorrência em um caminho que ela salvou.
- **Informação confiável:** em grupos de mensagem circulam fotos de enchentes de outros anos. Na Rota Seca cada ocorrência mostra há quanto tempo foi registrada, quantos vizinhos confirmaram e se a Defesa Civil validou, e deixa de valer depois de 3 horas.
- **Rede de apoio:** familiares cadastrados recebem os alertas graves e podem acompanhar o bairro de quem mora sozinho.
- **Onde esperar:** pontos de espera coberta e abrigos com status aberto ou fechado atualizado por agentes.

Objetivos

Objetivo geral

Disponibilizar uma API em Java que concentre alertas, ocorrências, pontos de apoio e contatos de confiança, para que a aplicação avise a pessoa certa, no lugar certo e a tempo.

Objetivos específicos

- CRUD completo das 14 tabelas do modelo de dados
- Regras de negócio na camada BO, separadas do acesso a dados
- Erros com status HTTP correto e mensagem clara
- Publicação em nuvem (Render) ligada ao Oracle da FIAP

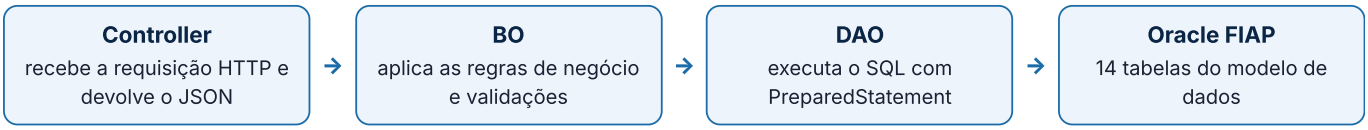
Público

Moradores de áreas que alagam (em especial idosos e quem depende de ônibus), seus familiares, agentes da Defesa Civil, que validam ocorrências e atualizam pontos de apoio, e administradores, que cadastram bairros e vias.

2

Arquitetura da API

O projeto segue o padrão visto em aula: Spring Boot com **JDBC puro** e separação em pacotes por responsabilidade. Cada requisição passa pelas camadas abaixo:



Quando algo dá errado, a camada que percebe o problema lança uma exceção (`RegraNegocioException`, `RecursoNaoEncontradoException`, `ExcecoesConexao` ou a `SQLException` do banco). O `TratadorExcecoes` transforma cada uma na resposta HTTP certa.

Pacotes e classes (73 classes)

Pacote	Responsabilidade	Classes
entities 14 classes	Domínio: uma classe por tabela, com atributos privados, construtores, getters/setters e toString	Alerta, Bairro, BairroAcompanhado, CaminhoSalvo, CaminhoTrecho, Confirmacao, ContatoConfianca, LeituraChuva, NoVia, Notificacao, Ocorrencia, PontoApoio, TrechoVia, Usuario
dao 16 classes	Acesso a dados com JDBC puro (PreparedStatement) e try-with-resources	AlertaDAO, AvisoCaminhoDAO, BairroAcompanhadoDAO, BairroDAO, CaminhoSalvoDAO, CaminhoTrechoDAO, ConfirmacaoDAO, ContatoConfiancaDAO, JdbcUtil, LeituraChuvaDAO, NoViaDAO, NotificacaoDAO, OcorrenciaDAO, PontoApoioDAO, TrechoViaDAO, UsuarioDAO
bo 17 classes	Regras de negócio e validações antes de chegar ao banco	AlertaBo, Autorizacao, BairroAcompanhadoBo, BairroBo, CaminhoSalvoBo, CaminhoTrechoBo, ConfirmacaoBo, ContatoConfiancaBo, LeituraChuvaBo, NoViaBo, NotificacaoBo, OcorrenciaBo, PontoApoioBo, Relogio, TrechoViaBo, UsuarioBo, Validacao
controller 16 classes	Endpoints REST (@RestController) e configuração de CORS	AlertaController, BairroAcompanhadoController, BairroController, CaminhoSalvoController, CaminhoTrechoController, ConfirmacaoController, ContatoConfiancaController, CorsConfig, LeituraChuvaController, NoViaController, NotificacaoController, OcorrenciaController, PontoApoioController, StatusController, TrechoViaController, UsuarioController
conexoes 1 classes	Fábrica de conexões com o Oracle (credenciais por variável de ambiente)	ConexaoFactory
excecoes 6 classes	Exceções próprias e tratamento global (@RestControllerAdvice)	AcessoNegadoException, ErroResposta, ExcecoesConexao, RecursoNaoEncontradoException, RegraNegocioException, TratadorExcecoes
dto 2 classes	Objetos de entrada e saída que não são tabelas	AvisoCaminho, ValidacaoRequest
(raiz) 1 classes	Classe principal que inicia o Spring Boot	RotaSecaApplication

Decisões técnicas

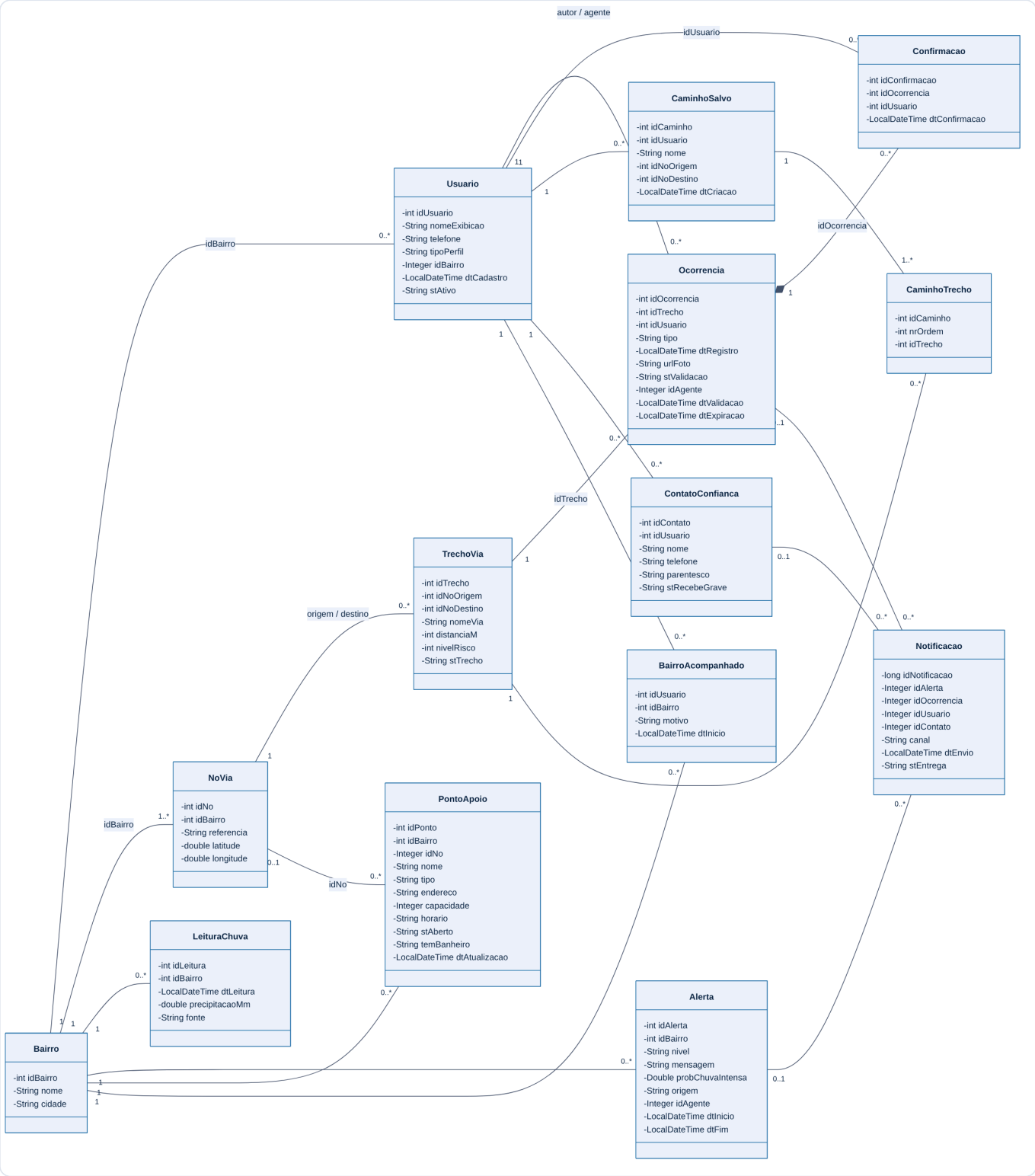
- **Conexão aberta e fechada em cada método** com *try-with-resources*, para não deixar conexões presas no servidor Oracle compartilhado da FIAP.
- **Senha fora do código:** o repositório é público, então usuário e senha do banco vêm das variáveis de ambiente `DB_USER` e `DB_PASSWORD`.
- **Operações oficiais protegidas** por credencial de agente (cabeçalho `X-Token-Agente`), com resposta 403 sem ela; a Defesa Civil é simulada nesta versão.
- **Horário de Brasília** calculado em Java (`Relogio`), porque o servidor do Render roda em UTC.
- **ID gerado pela API** quando o JSON não informa (próximo valor livre da tabela).
- **Deploy com Docker** (imagem Java 21), igual ao modelo da aula, com a porta lida da variável `PORT` do Render.

3 · DIAGRAMA DE CLASSES

10 pontos

3.1 Domínio: entidades e associações

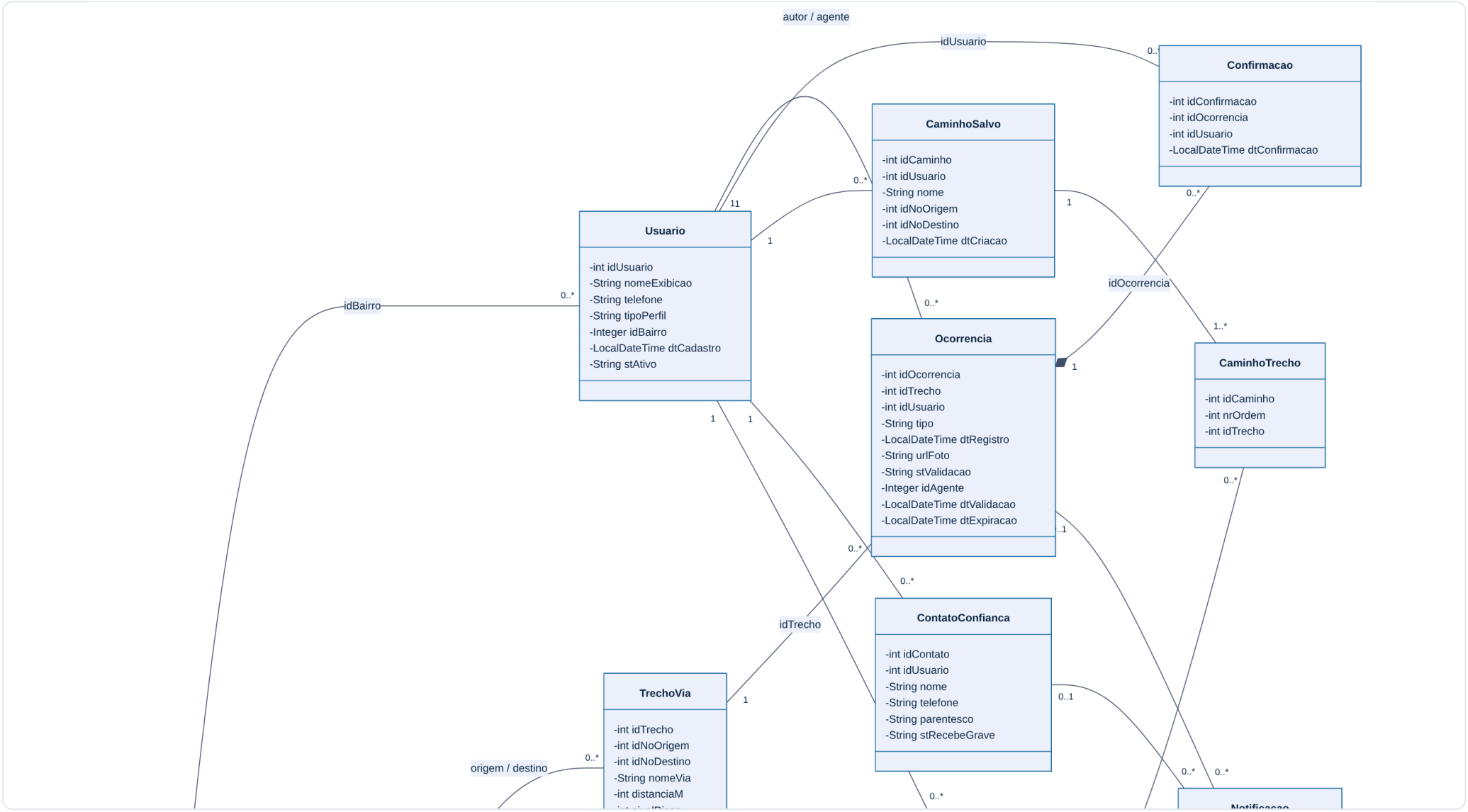
As 14 entidades do pacote `entities`, com seus atributos e tipos. Getters, setters, construtores e `toString` existem em todas e foram omitidos para caber na página.



3 · AMPLIAÇÃO

3.1 Domínio — parte 1 de 2

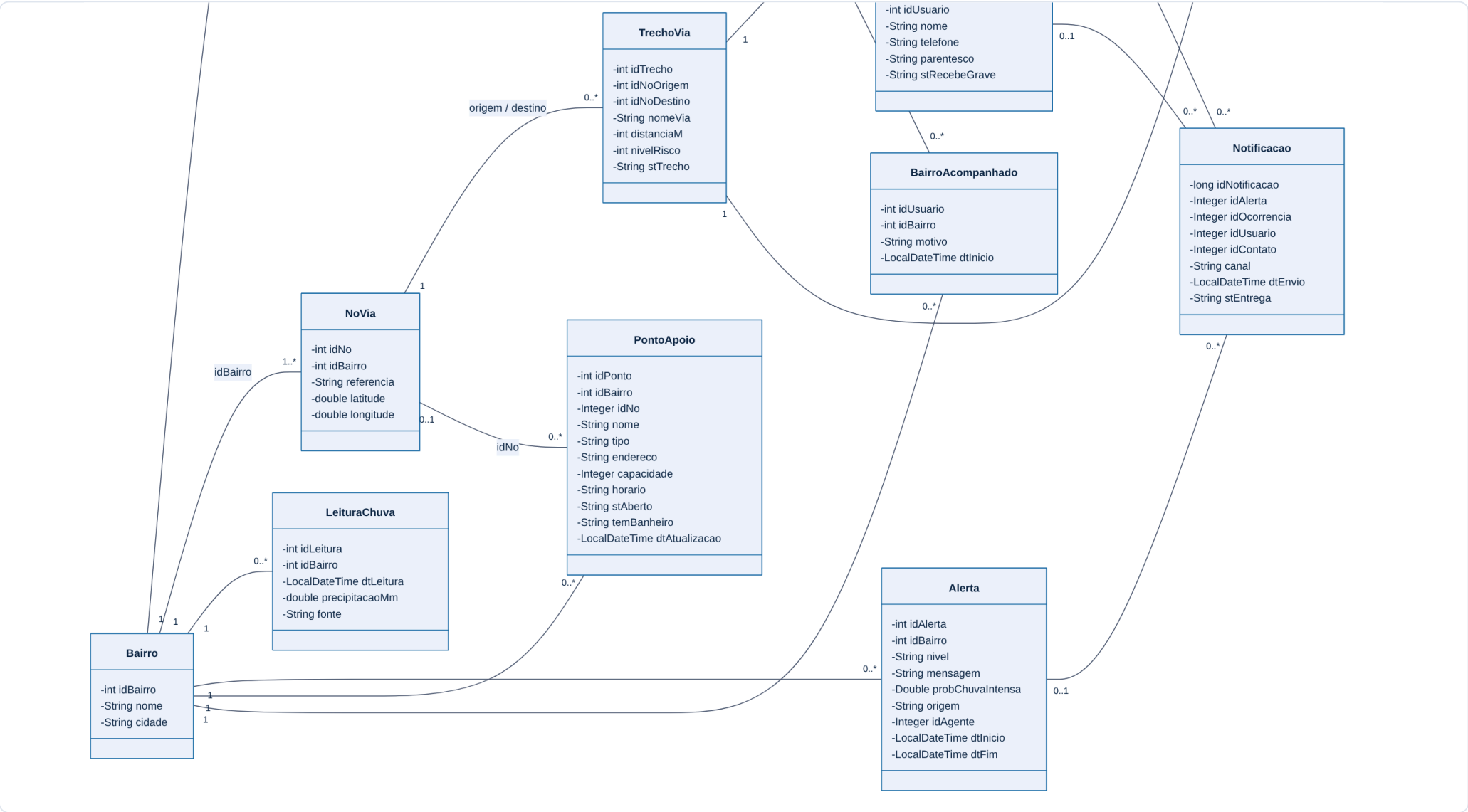
Ampliação do diagrama da página anterior; as partes se sobrepõem um pouco para nenhuma classe ficar cortada.



3 · AMPLIAÇÃO

3.1 Domínio — parte 2 de 2

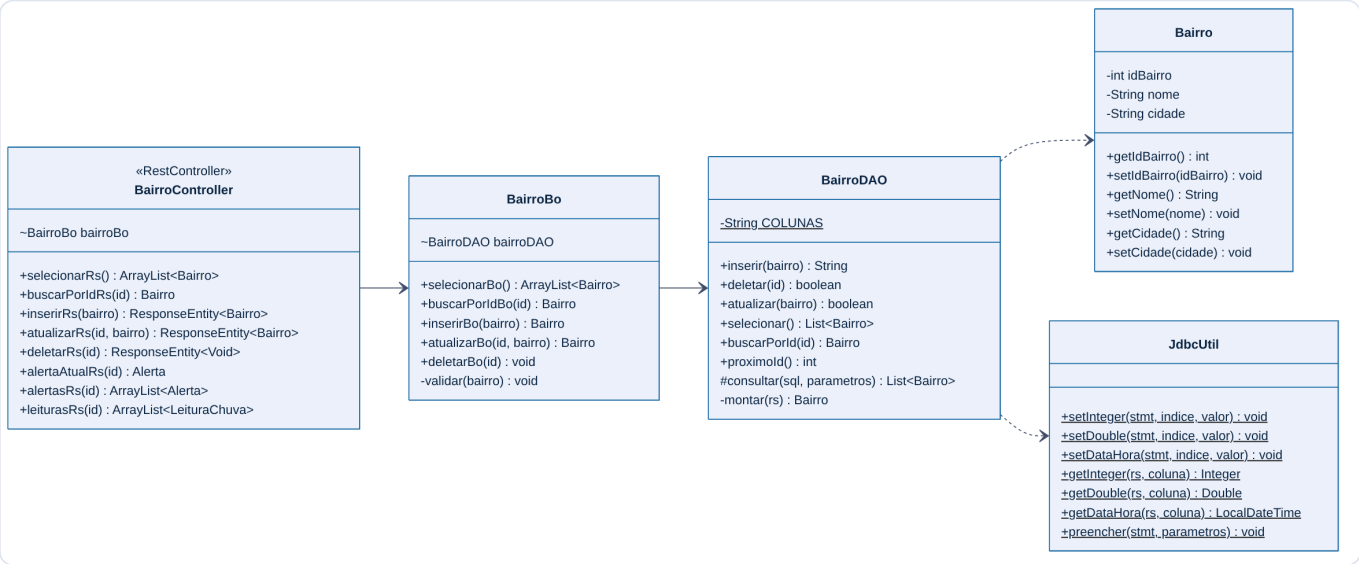
Ampliação do diagrama da página anterior; as partes se sobrepõem um pouco para nenhuma classe ficar cortada.



3 · DIAGRAMA DE CLASSES

3.2 Padrão Controller → BO → DAO → Entidade

Abaixo, o exemplo de **Bairro**. As demais entidades seguem o mesmo padrão de camadas e operações, com exceções justificadas pelas regras de negócio: a **Confirmacao** não tem PUT (é imutável depois de registrada) e as tabelas de chave composta (**CaminhoTrecho** e **BairroAcompanhado**) usam rotas aninhadas com as duas chaves.

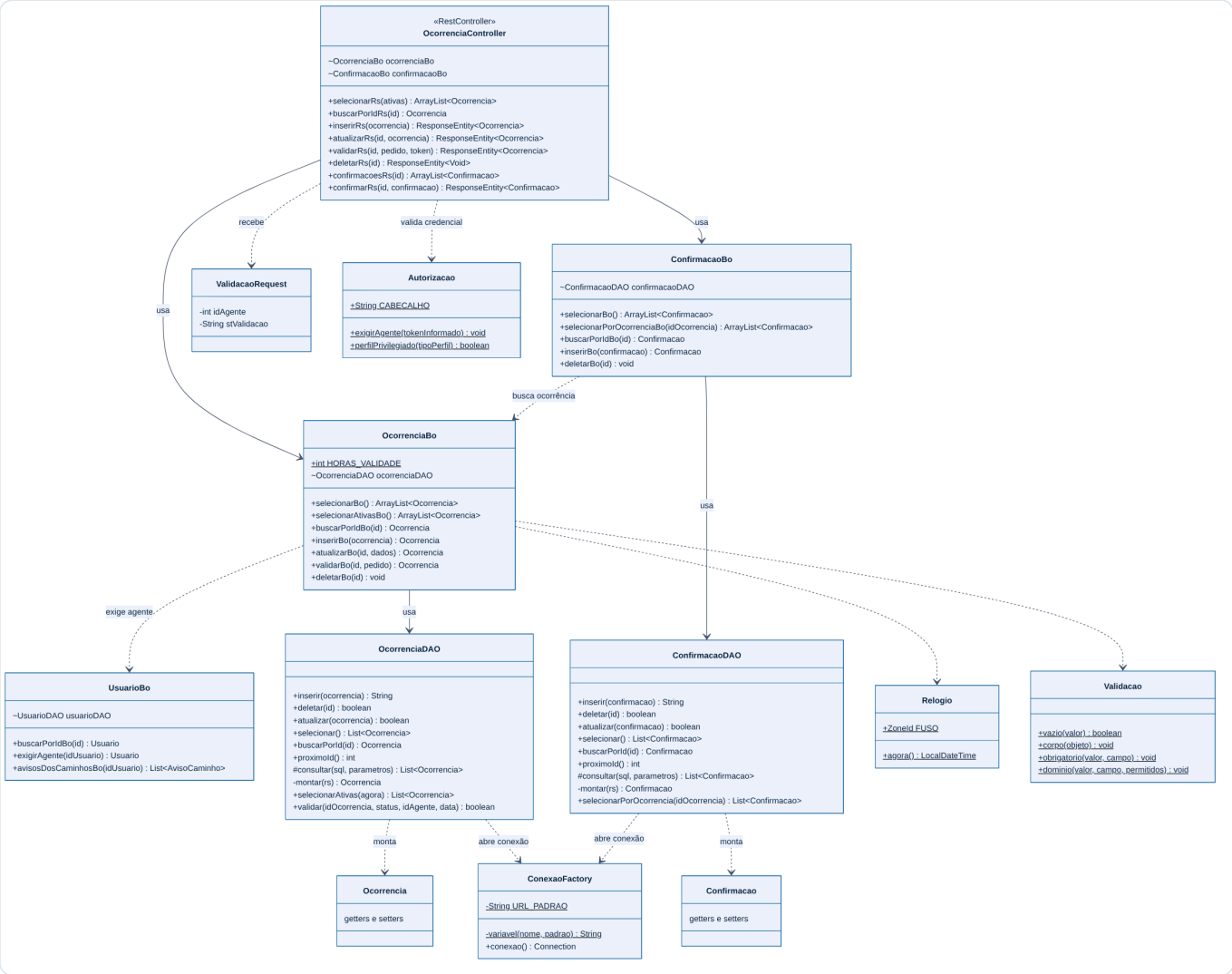


Métodos padrão

- **Controller:** `selecionarRs`, `buscarPorIdRs`, `inserirRs`, `atualizarRs`, `deletarRs`
- **BO:** os mesmos com sufixo `Bo` e um `validar()` privado com as regras da tabela
- **DAO:** `inserir`, `deletar`, `atualizar`, `selecionar`, `buscarPorId`, `proximoId` e consultas específicas

3.3 Fluxo principal: ocorrência, confirmação e validação

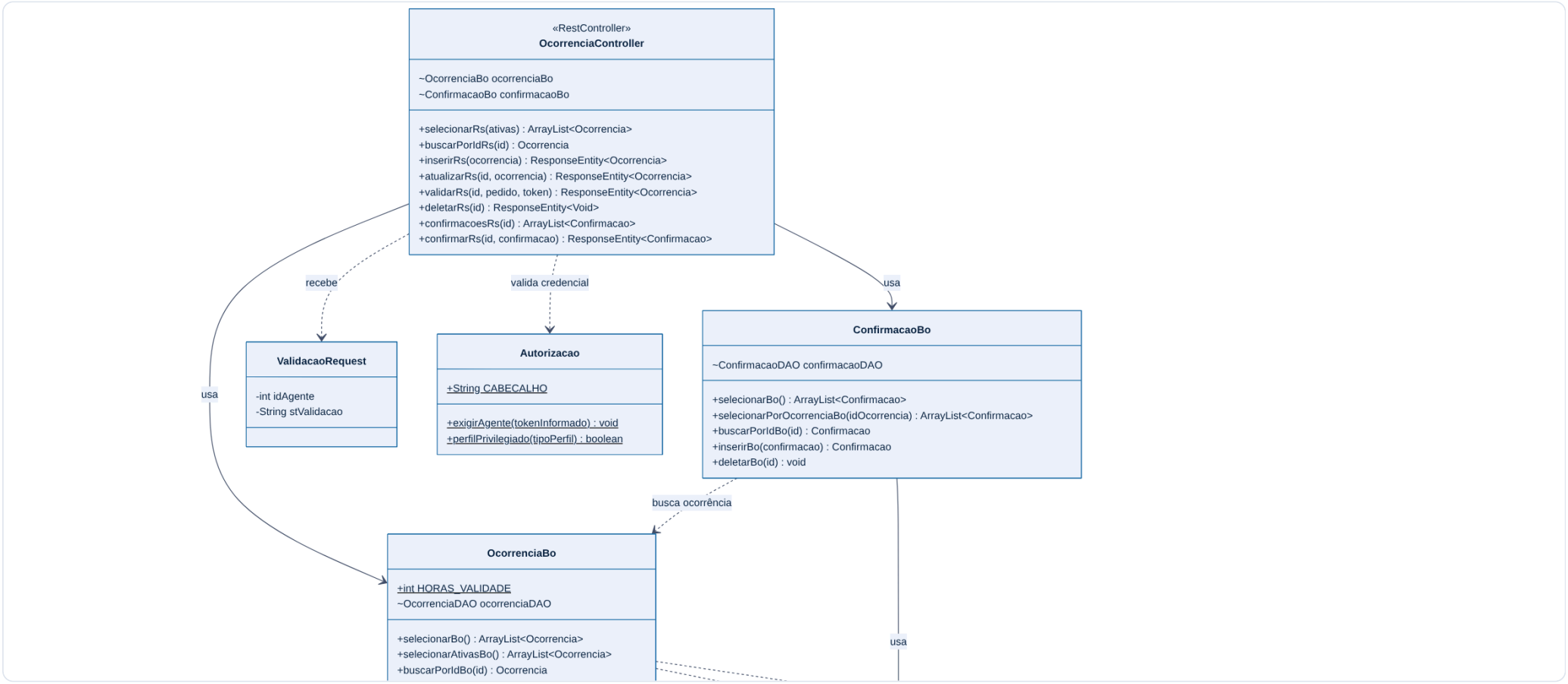
Classes envolvidas quando um morador reporta um alagamento, um vizinho confirma e um agente da Defesa Civil valida.



3 · AMPLIAÇÃO

3.3 Fluxo da ocorrência — parte 1 de 2

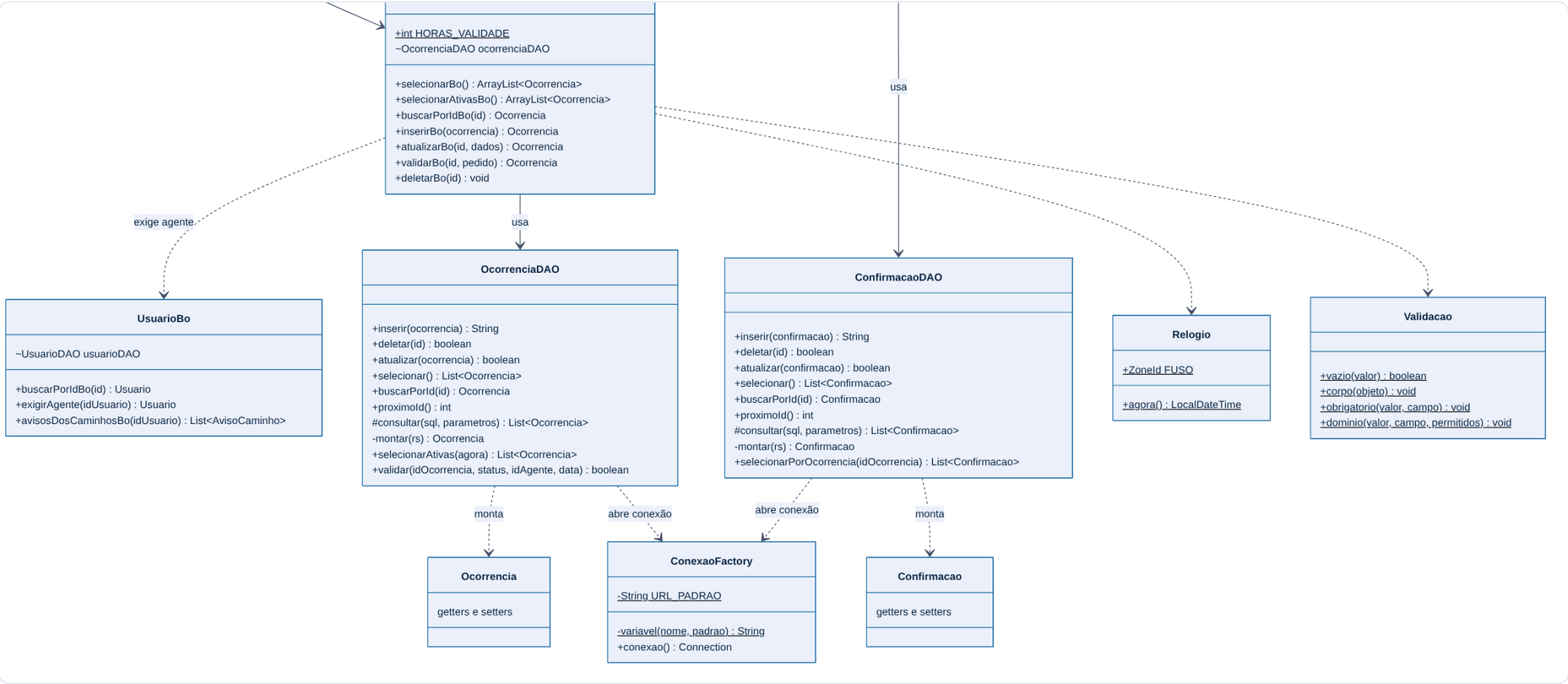
Ampliação do diagrama da página anterior; as partes se sobrepõem um pouco para nenhuma classe ficar cortada.



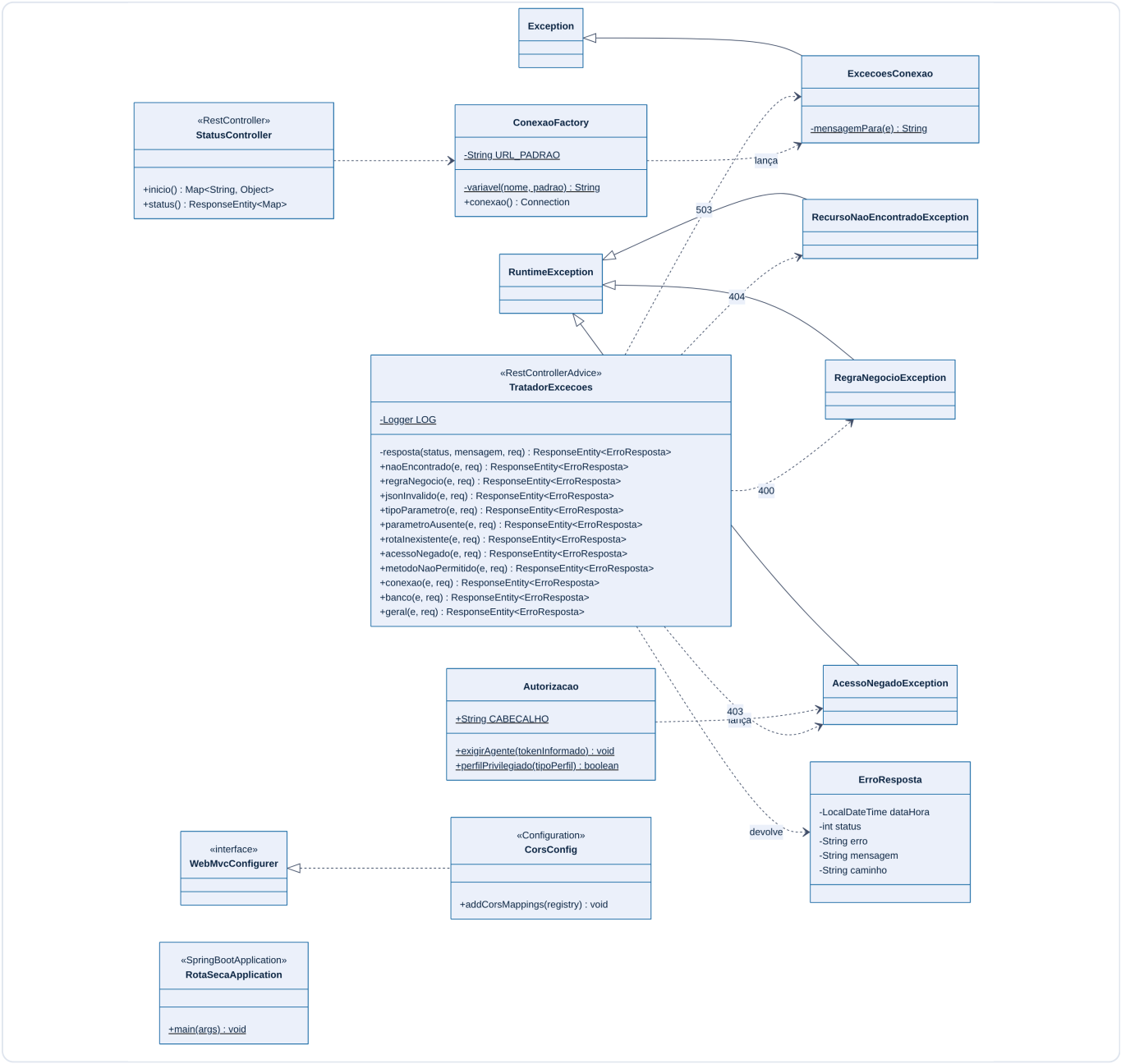
3 · AMPLIAÇÃO

3.3 Fluxo da ocorrência — parte 2 de 2

Ampliação do diagrama da página anterior; as partes se sobrepõem um pouco para nenhuma classe ficar cortada.



3.4 Exceções e infraestrutura



Os diagramas foram gerados a partir do próprio código-fonte do repositório, então refletem exatamente as classes, atributos e métodos implementados.

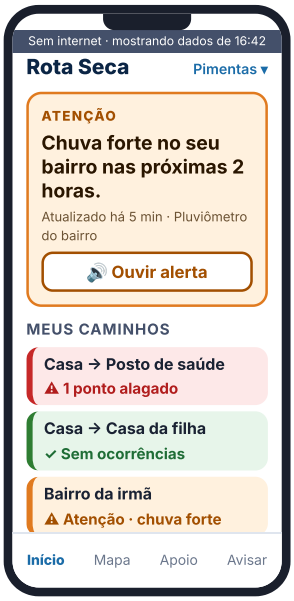
4

10 pontos

Protótipos de tela

Telas do aplicativo (as mesmas do documento de Agile) e os endpoints da API que alimentam cada uma.

Início



Alerta do bairro, caminhos salvos e bairro da irmã.

`GET /bairros/{id}/alerta-atual`
`GET /usuarios/{id}/avisos`
`GET /usuarios/{id}/bairros-acompanhados`

Alerta grave



Tela cheia, frase curta e aviso à família. Nesta versão o envio de push/SMS é simulado: a API registra a notificação no banco.

`POST /alertas`
`GET /usuarios/{id}/contatos`
`POST /notificacoes`

Mapa de ocorrências



Foto, há quanto tempo, confirmações e selo da Defesa Civil.

`GET /ocorrencias?ativas=true`
`GET /ocorrencias/{id}/confirmacoes`
`POST /ocorrencias/{id}/confirmacoes`

Avisar vizinhos



Reportar com um toque; foto opcional.

`POST /ocorrencias`
`PUT /ocorrencias/{id}/validacao`

Comparar trajetos



Trechos com ocorrência em cada caminho; sem prometer rota segura.

`GET /caminhos/{id}/trechos`
`GET /trechos-via`
`GET /ocorrencias?ativas=true`

Onde esperar



Espera coberta e abrigos com status aberto ou fechado.

`GET /pontos-apoio?aberto=S`
`PUT /pontos-apoio/{id}/status`

Tabela de endpoints

79 endpoints, lidos diretamente dos controllers. Base: https://rota-seca-api.onrender.com. Os erros seguem o padrão da seção 7.

Método	Caminho	O que faz	Sucesso	Erros
StatusController				
GET	`/`	Informações da API (nome, equipe, aviso e recursos)	200	—
GET	`/status`	Testa a conexão com o Oracle da FIAP	200	503
BairroController				
GET	`/bairros`	Lista todos os registros	200	404
GET	`/bairros/{id}`	Busca um registro pelo ID	200	404
POST	`/bairros`	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	`/bairros/{id}`	Atualiza o registro corpo JSON	200	400 · 404
DELETE	`/bairros/{id}`	Exclui o registro	204	404 · 409
GET	`/bairros/{id}/alerta-atual`	Alerta vigente do bairro (US01)	200	404
GET	`/bairros/{id}/alertas`	Histórico de alertas do bairro	200	404
GET	`/bairros/{id}/leituras-chuva`	Leituras de chuva do bairro	200	404
UsuarioController				
GET	`/usuarios`	Lista todos os registros	200	404
GET	`/usuarios/{id}`	Busca um registro pelo ID	200	404
POST	`/usuarios`	Cadastra um registro (ID gerado se omitido) corpo JSON · **X-Token-Agente se o perfil for AGENTE/ADMIN**	201	400 · 409 · 403
PUT	`/usuarios/{id}`	Atualiza o registro corpo JSON · **X-Token-Agente se o perfil for AGENTE/ADMIN**	200	400 · 404 · 403
DELETE	`/usuarios/{id}`	Exclui o registro **X-Token-Agente se o perfil for AGENTE/ADMIN**	204	404 · 409 · 403
GET	`/usuarios/{id}/avisos`	Avisos de ocorrências nos caminhos salvos do usuário (US02)	200	404
GET	`/usuarios/{id}/caminhos`	Caminhos salvos do usuário	200	404
GET	`/usuarios/{id}/contatos`	Contatos de confiança do usuário	200	404
BairroAcompanhadoController				
GET	`/usuarios/{idUsuario}/bairros-acompanhados`	Lista todos os registros	200	404
POST	`/usuarios/{idUsuario}/bairros-acompanhados`	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	`/usuarios/{idUsuario}/bairros-acompanhados/{idBairro}`	Atualizar o motivo	200	400 · 404
DELETE	`/usuarios/{idUsuario}/bairros-acompanhados/{idBairro}`	Exclui o registro	204	404 · 409
NoViaController				
GET	`/nos-via`	Lista todos os registros	200	404
GET	`/nos-via/{id}`	Busca um registro pelo ID	200	404
POST	`/nos-via`	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409

Método	Caminho	O que faz	Sucesso	Erros
PUT	`/nos-via/{id}`	Atualiza o registro corpo JSON	200	400 · 404
DELETE	`/nos-via/{id}`	Exclui o registro	204	404 · 409
TrechoViaController				
GET	`/trechos-via`	Lista todos os registros	200	404
GET	`/trechos-via/{id}`	Busca um registro pelo ID	200	404
POST	`/trechos-via`	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	`/trechos-via/{id}`	Atualiza o registro corpo JSON	200	400 · 404
DELETE	`/trechos-via/{id}`	Exclui o registro	204	404 · 409
OcorrenciaController				
GET	`/ocorrencias`	Selecionar (?ativas=true traz só as não descartadas e não expiradas) query: ativas	200	404
GET	`/ocorrencias/{id}`	Busca um registro pelo ID	200	404
POST	`/ocorrencias`	Inserir (reportar ocorrência) corpo JSON	201	400 · 409
PUT	`/ocorrencias/{id}`	Atualiza o registro corpo JSON	200	400 · 404
PUT	`/ocorrencias/{id}/validacao`	Validar (agente da Defesa Civil) corpo JSON · **exige X-Token-Agente**	200	400 · 404 · 403
DELETE	`/ocorrencias/{id}`	Exclui o registro	204	404 · 409
GET	`/ocorrencias/{id}/confirmacoes`	Confirmações da ocorrência	200	404
POST	`/ocorrencias/{id}/confirmacoes`	Confirmar ocorrência (outro morador viu a mesma coisa) corpo JSON	201	400 · 409
ConfirmacaoController				
GET	`/confirmacoes`	Lista todos os registros	200	404
GET	`/confirmacoes/{id}`	Busca um registro pelo ID	200	404
POST	`/confirmacoes`	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
DELETE	`/confirmacoes/{id}`	Exclui o registro	204	404 · 409
LeituraChuvaController				
GET	`/leituras-chuva`	Lista todos os registros	200	404
GET	`/leituras-chuva/{id}`	Busca um registro pelo ID	200	404
POST	`/leituras-chuva`	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	`/leituras-chuva/{id}`	Atualiza o registro corpo JSON	200	400 · 404
DELETE	`/leituras-chuva/{id}`	Exclui o registro	204	404 · 409
AlertaController				
GET	`/alertas`	Lista todos os registros	200	404
GET	`/alertas/{id}`	Busca um registro pelo ID	200	404
POST	`/alertas`	Cadastra um registro (ID gerado se omitido) corpo JSON · **exige X-Token-Agente**	201	400 · 409 · 403
PUT	`/alertas/{id}`	Atualiza o registro corpo JSON · **exige X-Token-Agente**	200	400 · 404 · 403

Método	Caminho	O que faz	Sucesso	Erros
DELETE	/alertas/{id}	Exclui o registro **exige X-Token-Agente**	204	404 · 409 · 403
PontoApoioController				
GET	/pontos-apoio	Selecionar (filtros opcionais: ?idBairro=1&aberto=S) query: idBairro	200	404
GET	/pontos-apoio/{id}	Busca um registro pelo ID	200	404
POST	/pontos-apoio	Cadastra um registro (ID gerado se omitido) corpo JSON · **exige X-Token-Agente**	201	400 · 409 · 403
PUT	/pontos-apoio/{id}	Atualiza o registro corpo JSON · **exige X-Token-Agente**	200	400 · 404 · 403
PUT	/pontos-apoio/{id}/status	Abrir ou fechar o ponto (agente): PUT /pontos-apoio/1/status?aberto=S&idAgente=5 query: aberto · **exige X-Token-Agente**	200	400 · 404 · 403
DELETE	/pontos-apoio/{id}	Exclui o registro **exige X-Token-Agente**	204	404 · 409 · 403
CaminhoSalvoController				
GET	/caminhos	Lista todos os registros	200	404
GET	/caminhos/{id}	Busca um registro pelo ID	200	404
POST	/caminhos	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	/caminhos/{id}	Atualiza o registro corpo JSON	200	400 · 404
DELETE	/caminhos/{id}	Exclui o registro	204	404 · 409
CaminhoTrechoController				
GET	/caminhos/{idCaminho}/trechos	Selecionar trechos do caminho	200	404
POST	/caminhos/{idCaminho}/trechos	Inserir trecho no caminho corpo JSON	201	400 · 409
PUT	/caminhos/{idCaminho}/trechos/{nrOrdem}	Atualizar o trecho de uma posição	200	400 · 404
DELETE	/caminhos/{idCaminho}/trechos/{nrOrdem}	Deletar uma posição	204	404 · 409
ContatoConfiancaController				
GET	/contatos	Lista todos os registros	200	404
GET	/contatos/{id}	Busca um registro pelo ID	200	404
POST	/contatos	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	/contatos/{id}	Atualiza o registro corpo JSON	200	400 · 404
DELETE	/contatos/{id}	Exclui o registro	204	404 · 409
NotificacaoController				
GET	/notificacoes	Lista todos os registros	200	404
GET	/notificacoes/{id}	Busca um registro pelo ID	200	404
POST	/notificacoes	Cadastra um registro (ID gerado se omitido) corpo JSON	201	400 · 409
PUT	/notificacoes/{id}	Atualiza o registro corpo JSON	200	400 · 404
DELETE	/notificacoes/{id}	Exclui o registro	204	404 · 409

Exemplo: reportar uma ocorrência

Requisição

```
POST /ocorrencias
Content-Type: application/json

{
  "idTrecho": 1,
  "idUsuario": 7,
  "tipo": "ALAGAMENTO"
}
```

Resposta 201 Created

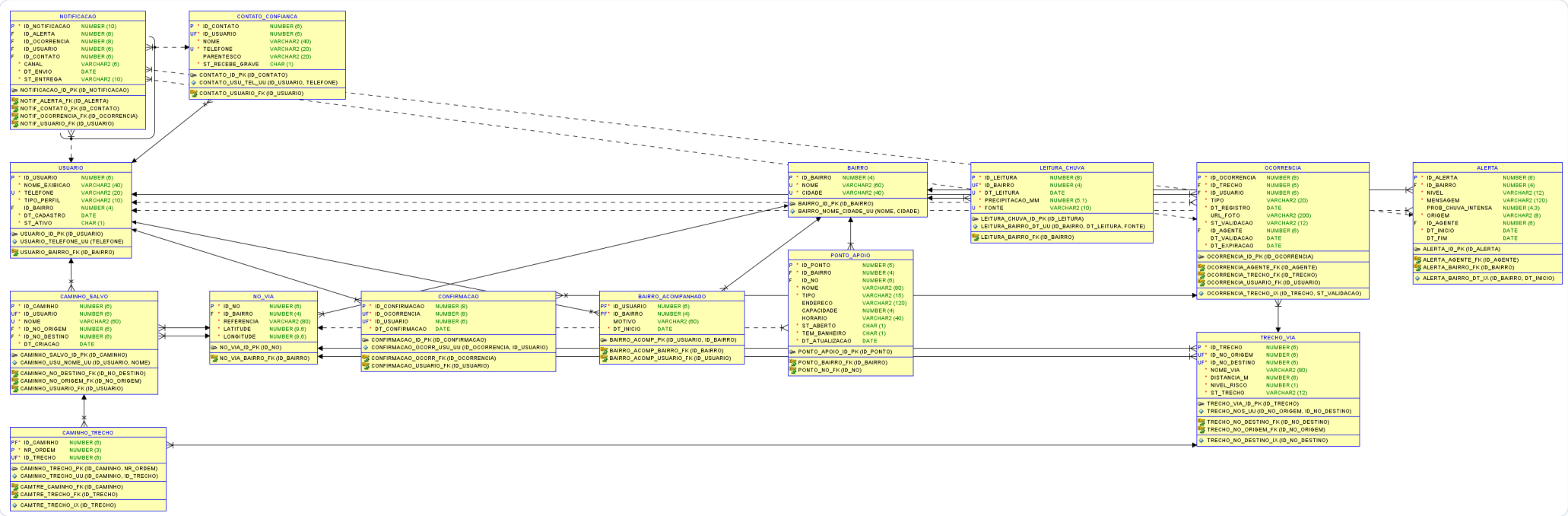

```
{
  "idOcorrencia": 7,
  "idTrecho": 1,
  "idUsuario": 7,
  "tipo": "ALAGAMENTO",
  "dtRegistro": "2026-11-09T16:16:30",
  "urlFoto": null,
  "stValidacao": "PENDENTE",
  "idAgente": null,
  "dtValidacao": null,
  "dtExpiracao": "2026-11-09T19:16:30"
}
```

6 · MODELO DE DADOS

parte dos 10 pontos

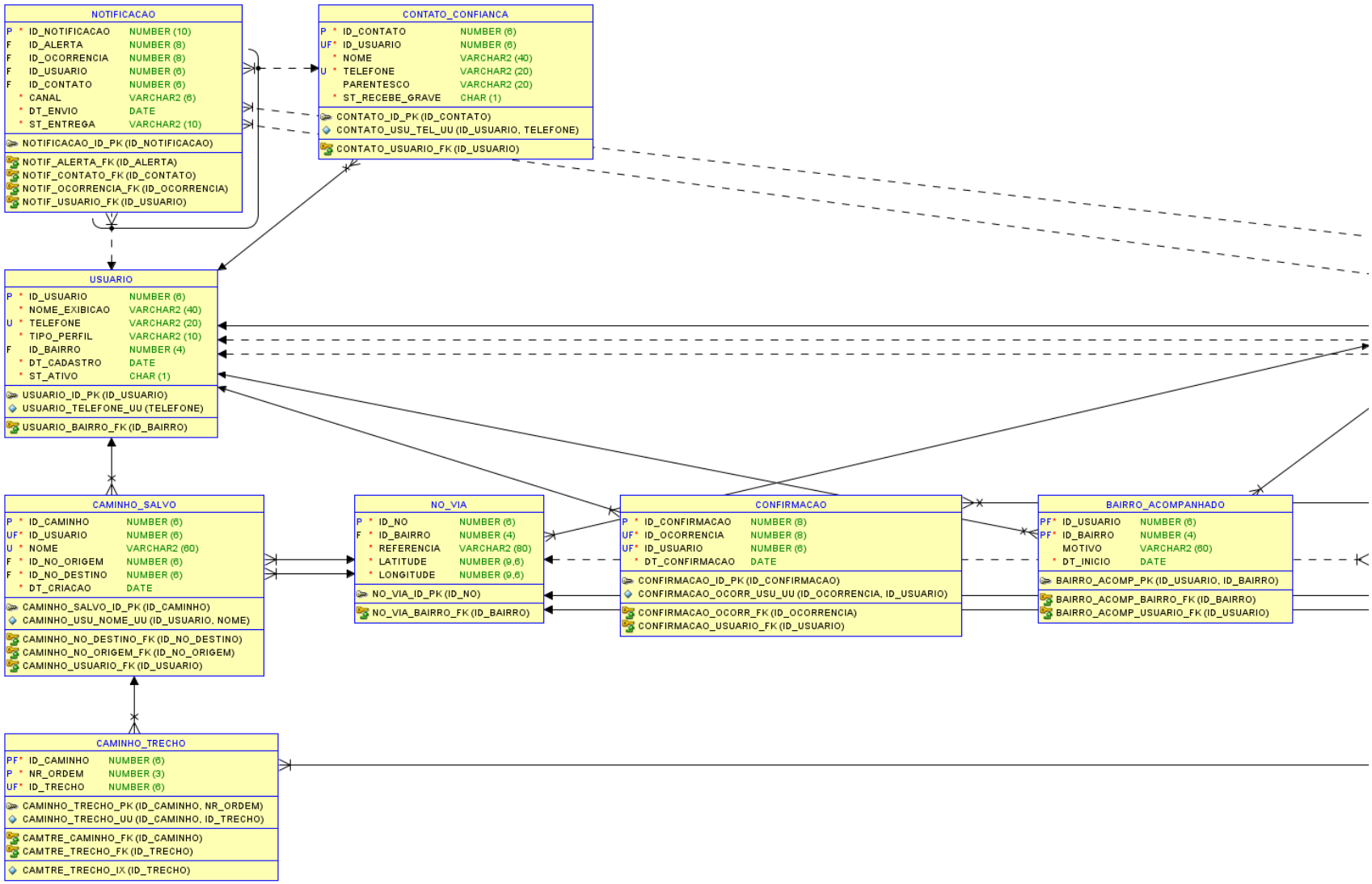
DER — Modelo Lógico (Oracle Data Modeler)

Mesmo modelo entregue em Database Design. Cada tabela corresponde a uma classe do pacote `entities`; ampliações nas duas páginas seguintes.



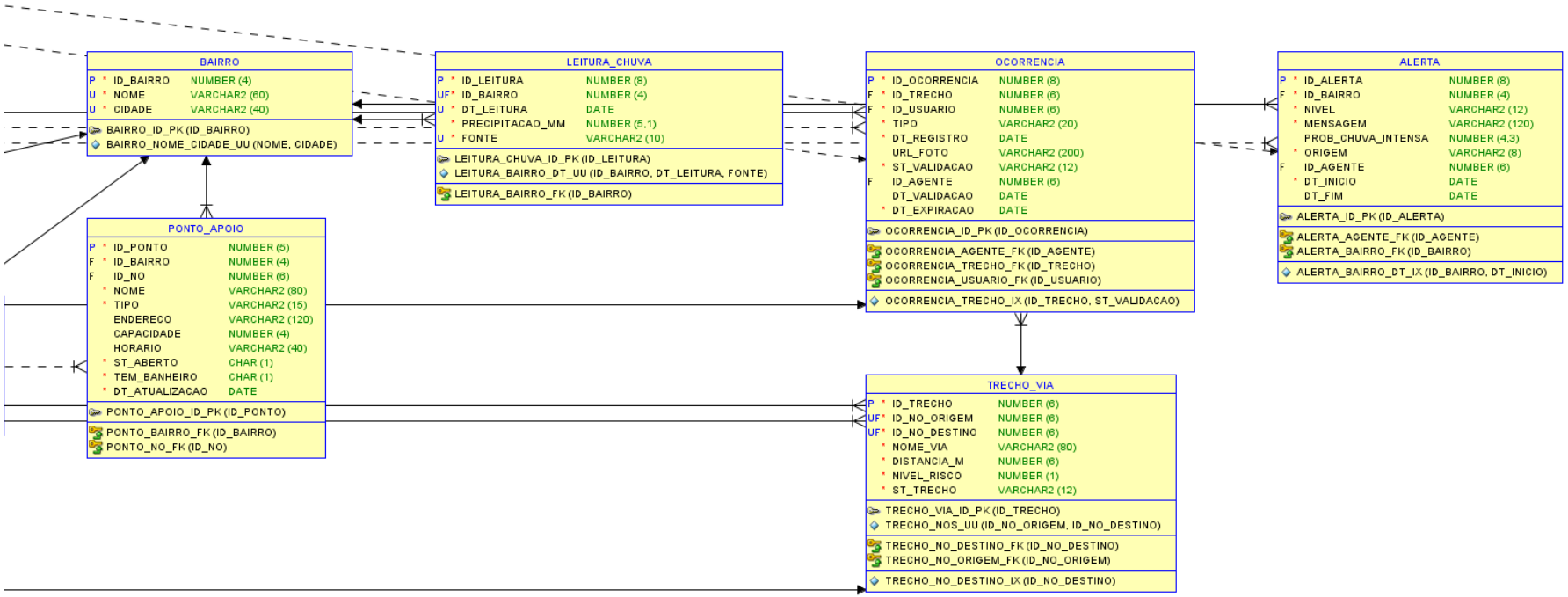
6 · AMPLIAÇÃO

DER — parte 1 de 2



6 · AMPLIAÇÃO

DER — parte 2 de 2



Regras de negócio e tratamento de exceções

Regras implementadas nos BOs

Classe	Regra
OcorrenciaBo	A API define a data de registro, o status `PENDENTE` e a validade de 3 horas; o app não consegue enviar uma ocorrência já "confirmada". Depois de validada, a ocorrência não pode mais ser editada.
ConfirmacaoBo	O autor não confirma a própria ocorrência; ocorrência descartada ou expirada não recebe confirmação; cada morador confirma uma única vez. Depois de registrada a confirmação é **imutável** (não há PUT): para corrigir, exclui-se e confirma-se de novo, passando por todas as regras.
Autorizacao	Validar ocorrência, emitir/alterar/excluir alerta, cadastrar ou alterar ponto de apoio, abrir/fechar ponto e criar/alterar/excluir usuários AGENTE ou ADMIN exigem a credencial de agente no cabeçalho `X-Token-Agente`. O valor fica na variável de ambiente `AGENTE_TOKEN`, fora do GitHub. A integração com a Defesa Civil é **simulada**: num sistema real, a credencial viria do login institucional do agente.
OcorrenciaBo / PontoApoioBo	Só usuário com perfil `AGENTE` valida ocorrências e abre ou fecha pontos de apoio.
AlertaBo	Alerta do modelo de Data Science sem nível recebe o nível pela probabilidade de chuva intensa: abaixo de 0,40 INFORMATIVO, abaixo de 0,75 ATENÇÃO, senão GRAVE. Alerta de origem AGENTE exige um agente válido. Mensagem com no máximo 120 caracteres.
NotificacaoBo	Exatamente uma origem (alerta ou ocorrência) e exatamente um destinatário (usuário ou contato). A API **registra** o aviso; o envio real de push ou SMS para celulares é simulado nesta versão.
TrechoViaBo / CaminhoSalvoBo	Origem e destino diferentes; distância maior que zero; nível de risco entre 0 e 5.
Todos	Campos obrigatórios e valores de domínio (ex.: `tipo` só aceita ALAGAMENTO, CORREGO_SUBINDO ou VIA_BLOQUEADA) são conferidos antes do SQL, com mensagem em português.

Tratamento de exceções

Situação	Exceção	Status
Registro não encontrado (GET, PUT ou DELETE com ID inexistente)	RecursoNaoEncontradoException	404
Regra de negócio, campo obrigatório ou valor fora do domínio	RegraNegocioException	400
JSON inválido, ID não numérico ou parâmetro ausente	exceções do Spring	400
ID referenciado não existe (FK, ORA-02291)	SQLException	400
Registro duplicado (UNIQUE, ORA-00001)	SQLException	409
Excluir registro que tem dependentes (ORA-02292)	SQLException	409
Operação de agente sem credencial válida	AcessoNegadoException	403
Método não permitido na rota (ex.: PUT em /confirmacoes)	exceção do Spring	405
Banco fora do ar, senha errada ou variáveis não configuradas	ExcecoesConexao	503
Qualquer outro erro (registrado no log)	Exception	500

Todas as respostas de erro têm o mesmo formato:

```
{"dataHora":"2026-11-09T16:16:41","status":409,"erro":"Conflict","mensagem":"Registro duplicado: já existe um
registro com estes dados","caminho":"/ocorrencias/7/confirmacoes"}
```

7 · EVIDÊNCIAS

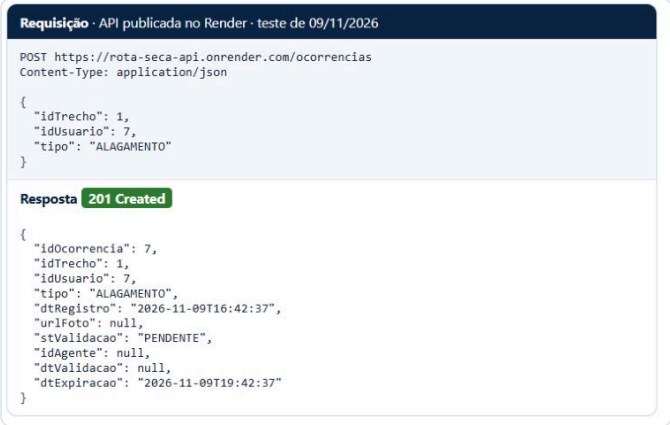
Testes realizados

80 cenários executados localmente com o banco carregado pelos scripts de Database Design (CRUD das 14 tabelas, regras de negócio, credencial de agente e erros), todos aprovados; o registro está em `docs/resultado_testes.txt`. Abaixo, capturas de testes feitos na **API publicada no Render**, ligada ao Oracle da FIAP:

GET /status — API e banco OK no Render



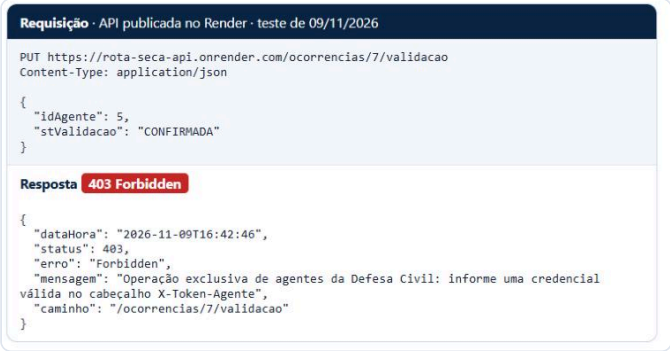
POST /ocorrencias — 201 Created, status e validade definidos pela API



Regra de negócio — autor tentando confirmar a própria ocorrência (400)



Credencial — validar ocorrência sem X-Token-Agente (403)



8

5 pontos

Instruções de execução e deploy

Links

Repositório GitHub	https://github.com/fajuniior/rota-seca-api
API publicada (Render)	https://rota-seca-api.onrender.com
Teste do banco	https://rota-seca-api.onrender.com/status → `{"api":"OK","banco":"OK"}`

Ferramentas e versões

Ferramenta	Versão	Uso
Java (JDK)	21	Linguagem
Spring Boot (spring-boot-starter-webmvc)	4.1.1	API REST, Tomcat embutido, JSON
Driver Oracle JDBC (ojdbc17)	23.26 (gerenciado pelo Spring Boot)	Conexão com o banco
Oracle Database	servidor FIAP (oracle.fiap.com.br:1521/orcl)	Banco de dados
Maven Wrapper	incluído no projeto (`mvnw`)	Build sem instalar o Maven
IntelliJ IDEA	2025 ou superior	IDE
Docker (imagem eclipse-temurin:21-jdk-alpine)	—	Empacotamento para o deploy
Render	plano Free	Hospedagem da API
Postman	qualquer	Testes (coleção em `docs/`)

Executar localmente

1. Clonar o repositório: `git clone https://github.com/fajuniior/rota-seca-api.git` e abrir a pasta no IntelliJ.
2. Criar as tabelas no Oracle da FIAP com os scripts `01_DDL_RotaSeca.sql` e `02_DML_RotaSeca.sql` (entrega de Database Design).
3. Em *Run › Edit Configurations › Environment variables*, informar `DB_USER=rmXXXXXX;DB_PASSWORD=suasenha;AGENTE_TOKEN=uma-credencial`.
4. Executar a classe `RotaSecaApplication` (ou `mvnw.cmd spring-boot:run` no Windows).
5. Abrir `http://localhost:8080/status` e conferir `"banco":"OK"`.
6. Importar `docs/RotaSeca.postman_collection.json` no Postman para testar GET, POST, PUT e DELETE.

Deploy no Render

1. *New › Web Service* e conectar o repositório do GitHub.
2. Language **Docker** (o `Dockerfile` está na raiz), branch `main`, plano **Free**.
3. Em *Environment Variables*, criar `DB_USER`, `DB_PASSWORD` e `AGENTE_TOKEN`.
4. Clicar em *Deploy*; a cada novo commit na `main` o Render publica de novo automaticamente.

Credencial de agente para testar a API publicada
Nas operações marcadas com *X-Token-Agente* na tabela de endpoints, envie o cabeçalho `X-Token-Agente: (valor informado ao professor na entrega)`. No Postman, basta preencher a variável `tokenAgente` da coleção. A credencial não fica no GitHub: só neste documento e na configuração do Render.

No plano gratuito do Render a API "dorme" depois de 15 minutos sem uso. A primeira requisição depois disso pode levar cerca de 1 minuto; as seguintes respondem normalmente.